# Especificación de Diseño de Software (WP.12)

**Estándar: ISO/IEC 29110-5-1-1:2025 (Tabla 24 - WP.12 Software Components Specification | Actividad SI.03)**

**Nombre del proyecto: SoftGames**

**Versión del documento: 2.0 (Avance al 20 de octubre de 2026 — Iteración 2 / Sprint 2)**

**Autores de casos de uso y diseño (Sprint 2): Rogelio Elian Loera Garay (3 casos de uso: CU-02, CU-03 y CU-04 + Todo el Frontend del Sprint 2 + Backend de CU-01 y CU-02) y Sebastian Del Hoyo Alvarado (1 caso de uso: CU-01 + Revisión y Pruebas)**

**Cliente: María De León Sigg**

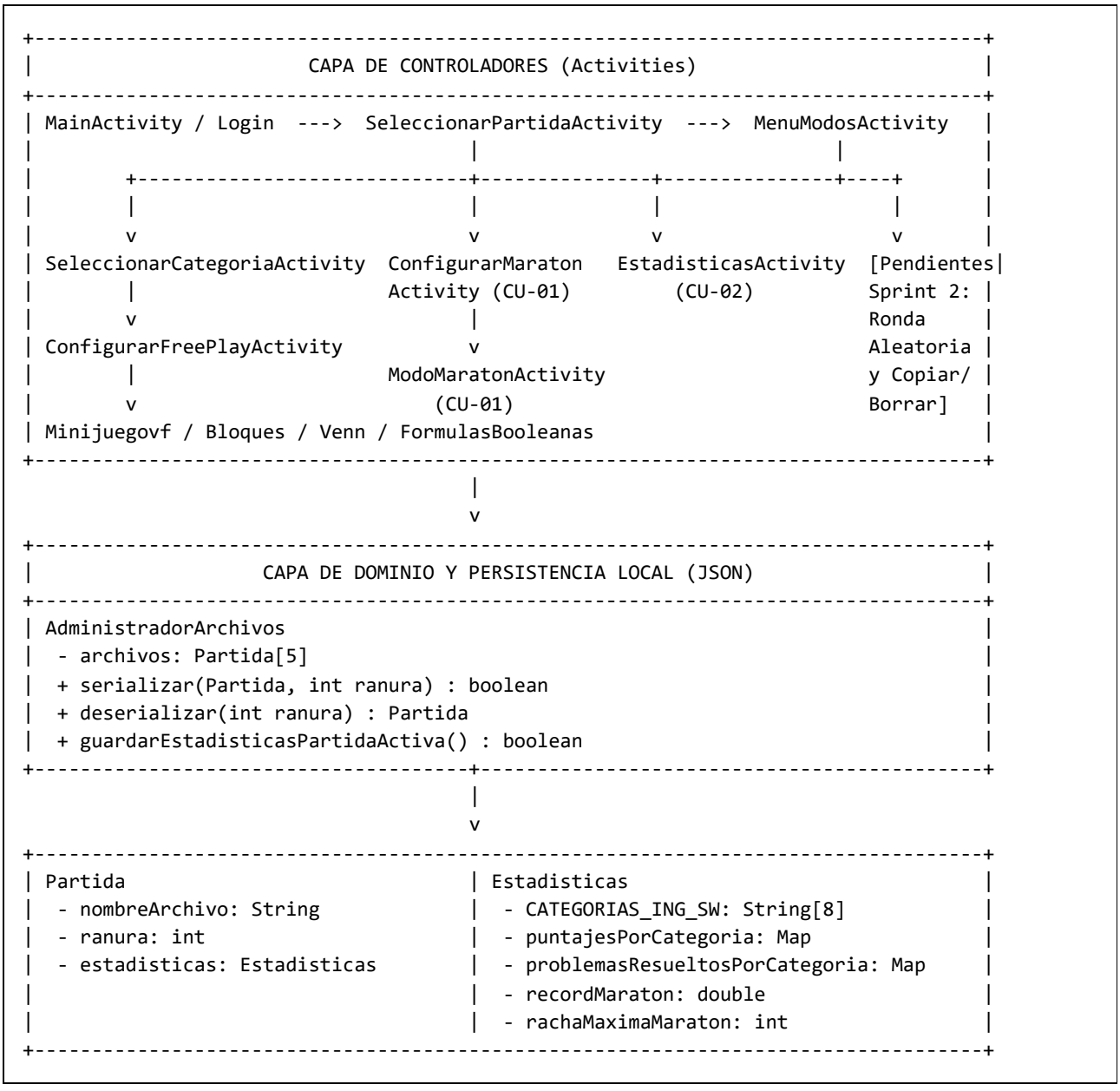
## 1.0 Propósito y Arquitectura de Alto Nivel (SI.03.03)

El presente documento especifica la arquitectura de software, los componentes construidos, las estructuras de datos locales (`.json`), las interfaces de usuario y la matriz de trazabilidad del proyecto **SoftGames** conforme a la Tabla 24 (**WP.12 Software components specification**) del estándar **ISO/IEC 29110-5-1-1:2025**, subsanando la observación de diseño de la auditoría de la Iteración 1.

- **Autoría de los Casos de Uso y Diseño de la Iteración 2:**
  - **Sebastian Del Hoyo Alvarado (1 caso de uso + Revisión/Pruebas):** Especificación y diseño funcional de **CU-01: Iniciar Modo Maratón**, revisión de código entre pares de CU-01 y CU-02, pruebas unitarias de CU-01 y asignación de Backend para CU-03.
  - **Rogelio Elian Loera Garay (3 casos de uso + Todo el Frontend + Backend CU-01/CU-02):** Especificación y diseño funcional de **CU-02: Registrar y consultar estadísticas de progreso por categoría**, **CU-03: Jugar Ronda Aleatoria** y **CU-04: Copiar o borrar archivo de guardado**, así como el diseño y construcción de todas las interfaces Frontend y el Backend de **CU-01** y **CU-02**.
- **Patrón arquitectónico seleccionado:** Arquitectura modular por capas basada en el patrón **Modelo-Vista-Controlador (MVC)** nativo para Android:
  - **Capa de Presentación (Vistas XML):** Archivos de diseño en `res/layout/` utilizando contenedores responsivos (`ScrollView`, `LinearLayout`, `ConstraintLayout`) y una paleta visual unificada de alto contraste académico (crema cálido `#F4F1DE`, arena `#D4C4A8`, azul marino `#0D1B2A` y verde salvia `#778D7A`).
  - **Capa de Control y Lógica de Juego (Controladores / Activities):** Clases Java que heredan de `AppCompatActivity`, encargadas de gestionar el ciclo de vida, el temporizador en tiempo real (`CountDownTimer`), la validación de respuestas por selección o arrastre de bloques (*drag and drop*) y la transición entre pantallas mediante `Intent`.
  - **Capa de Dominio y Persistencia Local (Modelo y Serialización JSON):** Clases de entidad (`Partida`, `Estadisticas`, bancos de reactivos por categoría) y el componente gestor `AdministradorArchivos`, que almacena y recupera hasta 5 archivos `.json` en el directorio interno privado del dispositivo (`Context.getFilesDir()`).

- **Tecnologías y entorno de desarrollo definidos:**
  - **Lenguaje y SDK:** Java (JDK 17 / Android SDK API 24+ compatible con Android 7.0 a Android 14).
  - **IDE y Construcción:** Android Studio con Gradle (Kotlin DSL).
  - **Persistencia de datos:** Biblioteca nativa `org.json` (`JSONObject`, `JSONArray`) sobre archivos locales en memoria interna sin dependencia de servidores externos.

## 2.0 Diagrama Estructural de Componentes y Clases




## 3.0 Diseño Detallado de Componentes de la Iteración 2 (Al 20 de octubre)

### 3.1 Caso de Uso 1 (Sprint 2): Iniciar Modo Maratón

- **Autor del Caso de Uso:** Sebastian Del Hoyo Alvarado | **Autor de la Codificación (Frontend y Backend):** Rogelio Elian Loera Garay.
- **Componentes construidos:** `ConfigurarMaratonActivity.java`, `ModoMaratonActivity.java`, `activity_configurar_maraton.xml` y `activity_modo_maraton.xml`.
- **Lógica de diseño implementada:**
    1. El usuario visualiza las **8 categorías de Ingeniería de Software** (*Programación, Introducción a la ingeniería de software, Lógica, Matemáticas discretas, Ingeniería de requerimientos, Análisis y diseño orientado a objetos, Programación orientada a objetos y Análisis y diseño de algoritmos*) mediante casillas seleccionables, además de un interruptor para el **Modo General** (que marca todas las categorías y todos los minijuegos).
    2. Si el usuario intenta iniciar sin marcar ninguna categoría, se dispara un aviso de validación (Flujo alternativo 2.1).
    3. En `ModoMaratonActivity` se inicializan: `erroresAcumulados = 0` (máximo 3 errores permitidos), `rachaActual = 0`, `puntajeTotal = 0.0` y arreglos asociativos para contabilizar los **puntos acumulados y la cantidad de problemas resueltos en cada categoría seleccionada**.
    4. La dificultad escala dinámicamente según la racha de aciertos: de 0 a 2 aciertos se presentan reactivos de dificultad *Fácil*, de 3 a 5 aciertos dificultad *Normal*, y a partir de 6 aciertos dificultad *Difícil*.
    5. Al acumular 3 errores (por respuesta incorrecta o por agotar el `CountDownTimer` del turno), la partida finaliza mostrando el diálogo/pantalla de resumen con el puntaje total, la racha lograda y el desglose de puntos y problemas resueltos por cada categoría seleccionada, guardando automáticamente el récord en el archivo `.json` activo.

### 3.2 Caso de Uso 2 (Sprint 2): Registrar y consultar estadísticas de progreso por categoría

- **Autor del Caso de Uso y de la Codificación (Frontend y Backend):** Rogelio Elian Loera Garay.
- **Componentes construidos:** `EstadisticasActivity.java`, `activity_estadisticas.xml`, `Estadisticas.java` y `AdministradorArchivos.java`.
- **Lógica de diseño implementada:**
    1. Al concluir cualquier ronda o modo de juego, el sistema invoca la actualización de `Estadisticas` sobre la partida activa y ejecuta `AdministradorArchivos.serializar()` en memoria interna.
    2. Si ocurre una excepción de entrada/salida (`IOException`), el sistema conserva los datos en el objeto en memoria de la sesión activa y notifica al usuario (Flujo alternativo 3.1).
    3. Al abrir `EstadisticasActivity`, se deserializa el archivo `.json` activo y se genera dinámicamente la tarjeta de resumen general (Promedio General, Récord de Maratón y

Categorías jugadas de 8) seguida de las tarjetas de las **8 categorías de Ingeniería de Software**.

4. Si una categoría aún no registra partidas, se muestra con puntaje `0.0 pts` y la leyenda *"Sin partidas registradas aún"* (Flujo alternativo 2.1).

### 3.3 Casos de Uso 3 y 4 (Sprint 2): Jugar Ronda Aleatoria y Copiar o borrar archivo de guardado

- **Autor de ambos Casos de Uso y su Diseño Funcional / Frontend:** Rogelio Elian Loera Garay.
- **Especificación de diseño lista para construcción (21 al 29 de octubre):**
  - **CU-03 (Jugar Ronda Aleatoria):** Selección aleatoria de reactivos combinando las 8 categorías de ingeniería de software, minijuegos y niveles de dificultad (Fácil, Normal, Difícil) sin requerir configuración manual previa, registrando el promedio final en el archivo `.json` activo.
  - **CU-04 (Copiar o borrar archivo de guardado):** Cuadro de diálogo de gestión sobre las ranuras ocupadas en `SeleccionarPartidaActivity` con métodos `copiarArchivo(origen, destino, nuevoNombre)` y `borrarArchivo(ranura)` dentro de `AdministradorArchivos`.

### 3.4 Especificación de Estructura de Datos Local (Archivo JSON)

```
{
  "nombrePartida": "PARTIDA_1",
  "ranura": 0,
  "recordMaraton": 74.8,
  "rachaMaximaMaraton": 9,
  "categorias": {
    "Programación": { "mejorPuntaje": 9.1, "problemasResueltos": 12 },
    "Introducción a la ingeniería de software": { "mejorPuntaje": 8.0, "problemasResueltos":
6 },
    "Lógica": { "mejorPuntaje": 8.6, "problemasResueltos": 10 },
    "Matemáticas discretas": { "mejorPuntaje": 0.0, "problemasResueltos": 0 },
    "Ingeniería de requerimientos": { "mejorPuntaje": 7.5, "problemasResueltos": 5 },
    "Análisis y diseño orientado a objetos": { "mejorPuntaje": 0.0, "problemasResueltos": 0
},
    "Programación orientada a objetos": { "mejorPuntaje": 8.8, "problemasResueltos": 8 },
    "Análisis y diseño de algoritmos": { "mejorPuntaje": 0.0, "problemasResueltos": 0 }
  }
}
```

## 4.0 Matriz de Trazabilidad (Requerimientos - Casos de Uso - Diseño - Código)

| Iteración | Caso de Uso y Autor | Requerimiento Funcional Cubierto | Clases y Layouts en Código Fuente |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Iteración 1 | Crear archivo de guardado | Creación y serialización inicial de hasta 5 archivos `.json` locales. | `SeleccionarPartidaActivity.java`, `AdministradorArchivos.java`, `Partida.java` |
| Iteración 1 | Seleccionar minijuego por categoría | Selección en modo Free Play entre las 8 categorías de ingeniería de software, dificultad y cantidad de preguntas (1-20). | `SeleccionarCategoriaActivity.java`, `ConfigurarFreePlayActivity.java` |
| Iteración 1 | Solución de minijuego | Temporizador visible y evaluación del 1.0 al 10.0 por velocidad y precisión (0 pts si falla o agota tiempo). | `MinijuegovfActivity.java`, `MinijuegoBloquesActivity.java`, `MinijuegoVennActivity.java`, `FormulasBooleanasActivity.java` |
| Iteración 1 | Pausar y Ajustes | Congelamiento de temporizador activo, ocultamiento de pregunta y menú de ajustes/salir. | `MenuPausaDialog.java`, `AjustesActivity.java` |
| **Iteración 2 (Completado)** | **CU-01: Iniciar Modo Maratón** (Diseño CU: Sebastian Alvarado \| Frontend y Backend: Elian Loera) | Selección de categorías o Modo General, dificultad progresiva, límite de 3 errores y resumen de puntos y problemas por categoría. | `ConfigurarMaratonActivity.java`, `ModoMaratonActivity.java`, `activity_configurar_maraton.xml`, `activity_modo_maraton.xml` |
| **Iteración 2 (Completado)** | **CU-02: Registrar y consultar estadísticas por categoría** (Diseño CU, Frontend y Backend: Elian Loera) | Serialización automática en `.json` y consulta de puntajes en las 8 categorías de ingeniería de software y sus minijuegos. | `EstadisticasActivity.java`, `Estadisticas.java`, `AdministradorArchivos.java`, `activity_estadisticas.xml` |
| Iteración 2 (Diseño listo; código pendiente) | **CU-03: Jugar Ronda Aleatoria** (Diseño CU y Frontend: Elian Loera \| Backend: Sebastian Alvarado) | Generación de partida rápida combinando categorías, minijuegos y dificultades al azar. | Diseño terminado por Elian Loera; codificación programada en `MenuModosActivity` / `RondaAleatoriaActivity`. |

| | | | |
|---|---|---|---|
| Iteración 2 (Diseño listo; código pendiente) | **CU-04: Copiar o borrar archivo de guardado** (Diseño CU y Frontend: Elian Loera \| Backend: Diego Alvarado) | Clonación de archivo `.json` en ranura libre o eliminación previa confirmación. | Diseño terminado por Elian Loera; codificación programada en `SeleccionarPartidaActivity` y `AdministradorArchivos`. |